25.1.2023

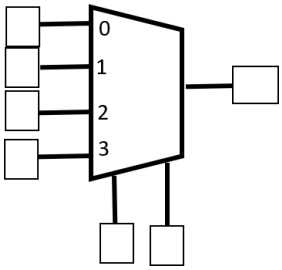
EE2003 and CSE 3017 DIGITAL DESIGN FINAL EXAM

Name:

CLOSED BOOK EXAM. ONE PAGE CHEAT SHEET IS ALLOWED. NO ELECTRONIC EQUIPMENT.
PLEASE PROVIDE YOUR ANSWERS IN DESIGNATED PLACES.

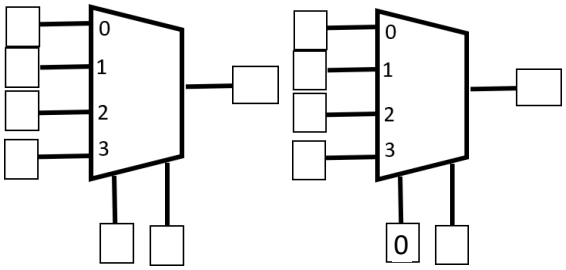
Q1) (20 points) We would like to form an SR latch by using multiplexers. Therefore,

a-) (10 points) Form the SR latch with the given 4 to 1 multiplexer only. Only the q output is needed for the SR latch. When SR={1,1} the latch should give q=1.

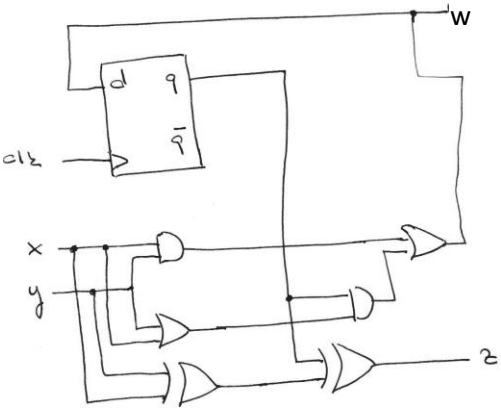


---

b-) (10 points) Extend the above design such that when SR={1,1} the latch should give $q = \bar{q}$. Use the second latch for this purpose. Now, the new SR latch will have outputs as q and $\bar{q}$.



Q2) (35 points) Consider the below sequential circuit.

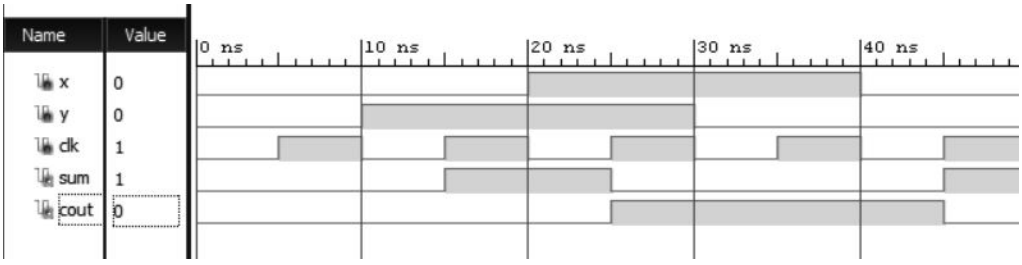


a-(10 points) Form the state and output table for this circuit.

q[n]	x	y	q[n+1]	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

---

b-) (10 points) What will be the output (w, z) of this circuit to the input given below. Initially q=0.



c-) (10 points) Plot the state diagram for the circuit.

d-) (5 points) What is the usage area (aim) of this circuit?

Sequential full adder.

EXTRA

```
module full_adder_sequential(x,y,clk,sum,cout);

input x,y,clk;
output reg sum,cout;

initial
begin
   sum=0;
   cout=0;
end

always @ (posedge clk)
begin
   sum  <= x^y^cout;
   cout <= x&y|cout&(x|y);
end

endmodule


`timescale 1ns / 1ps
module full_adder_sequential_tb;

reg x, y, clk;
wire sum, cout;

full_adder_sequential UUT(x,y,clk,sum,cout);

initial begin
clk = 0;
forever
#5 clk = ~clk;
end

initial
begin

x = 1'b0; y = 1'b0;
#10 x = 1'b0; y = 1'b1;
#10 x = 1'b1; y = 1'b1;
#10 x = 1'b1; y = 1'b0;
#10 x = 1'b0; y = 1'b0;

end

endmodule
```

Q3) (25 points) We will form a parity generator for 8-bit data. Check the below Verilog description formed for this purpose and fill in the empty lines. Note, each line requires one statement.

```
module parity_generator_sequential(input_bit,clk,start_bit,parity_type,parity_bit,count);

input input_bit,clk,start_bit,parity_type;
output reg parity_bit;
output reg [3:0] count;

reg parity_even=1'b0;
reg parity_odd=1'b0;
reg start_count=1'b0;

initial
parity_bit=1'b0;

always @ (negedge start_bit)
begin
count=4'b0000;
start_count=1'b1;
end

//this operation will be performed at every clock cycle and when the start count bit is set
always @ (posedge clk && start_count)                                    (5 points)
begin
if (count<4'b1000)
  begin
   count <= count+4'b0001;                                               (5 points)
   parity_even = parity_even^input_bit;                                  (5 points)
   parity_odd =  ~parity_even;                                           (5 points)
   parity_bit <= (parity_odd & parity_type) | (parity_even & ~parity_type);   (5 points)
  end
else
  begin
  start_count<=1'b0;
  count<=4'b0000;
  end
end

endmodule
```

EXTRA

```
`timescale 1ns / 1ps

module parity_generator_sequential_tb;

reg input_bit,clk,start_bit,parity_type;

wire parity_bit;
wire [3:0] count;

parity_generator_sequential UUT(input_bit,clk,start_bit,parity_type,parity_bit,count);

initial begin
#2 clk=0;
forever
#2 clk=~clk;
end

initial
begin
input_bit = 1'b0;
```

```
parity_type = 1'b1;

  start_bit= 1'b1;
#2 start_bit= 1'b0;

#4 input_bit = 1'b1;
#4 input_bit = 1'b0;
#4 input_bit = 1'b0;
#4 input_bit = 1'b1;
#4 input_bit = 1'b0;
#4 input_bit = 1'b1;
#4 input_bit = 1'b1;
#4 input_bit = 1'b0;

#2 parity_type = 1'b0;

  start_bit= 1'b1;
#2 start_bit= 1'b0;

#4 input_bit = 1'b1;
#4 input_bit = 1'b0;
#4 input_bit = 1'b0;
#4 input_bit = 1'b1;
#4 input_bit = 1'b0;
#4 input_bit = 1'b1;
#4 input_bit = 1'b1;
#4 input_bit = 1'b0;


end
endmodule
```

Q4) (20 points) Use dataflow modeling to form an SR latch with control input in Verilog.

```
module SR_latch_control(s,r,c,q,qn);

input s,r,c;
output q, qn;

assign q=~((r&c)|qn);
assign qn=~((s&c)|q);

endmodule

module SR_latch_control_tb;

reg s, r, c;
wire q, qn;

SR_latch_control UUT(s,r,c,q,qn);

initial begin
#5 c=1'b1; s=1'b1; r=1'b0;
#5 c=1'b1; s=1'b0; r=1'b0;
#5 c=1'b0; s=1'b0; r=1'b1;
#5 c=1'b0; s=1'b0; r=1'b0;

#5 c=1'b1; s=1'b0; r=1'b1;
#5 c=1'b1; s=1'b0; r=1'b0;
#5 c=1'b0; s=1'b1; r=1'b0;
#5 c=1'b0; s=1'b0; r=1'b0;

#5 c=1'b1; s=1'b1; r=1'b1;
```

```
#5 c=1'b1; s=1'b0; r=1'b0;
#5 c=1'b0; s=1'b1; r=1'b1;
#5 c=1'b0; s=1'b0; r=1'b0;
end

endmodule
```